

Stored Cross-Site Scripting Vulnerability

Stored XSS is a type of Cross-Site Scripting vulnerability where an attacker injects malicious scripts (typically JavaScript) into a web application, which stores them persistently on the server (e.g., in a database). These scripts are then served and executed in the browsers of unsuspecting users when they access the affected content. Unlike transient variants, the payload remains embedded, affecting multiple victims without repeated attacker involvement.

Key characteristics:

- **Persistent Nature:** Payloads are saved in forums, comments, profiles, or logs.
- **JavaScript-Centric:** Exploits often use JS to manipulate sessions or steal data.
- **OWASP Classification:** Falls under Injection flaws in the OWASP Top 10, specifically A03: Injection.

Related concepts include Blind Stored XSS, where the payload triggers in admin panels unseen by the attacker.

History:

Past:

Stored XSS emerged alongside early dynamic web apps in the late 1990s, but gained notoriety in the 2000s as user-generated content proliferated. The term XSS was coined in 2000, with stored variants highlighted in CERT advisories. A landmark event was the 2005 Samy Worm on MySpace, a stored XSS payload that self-propagated, infecting over a million profiles in hours.

Early exploits targeted blogs and social sites; detection involved manual reviews or basic scripts scanning for unescaped inputs.

Present:

In 2026, stored XSS dominates plugin vulnerabilities, with numerous CVEs in WordPress (e.g., Form Maker by 10Web, CVE-2026-24392 in HurryTimer) and apps like Docmost (CVE-2026-24045). Tools emphasize automated scanning:

- **Detection Tools:**
 - Burp Suite: Active scans for stored inputs; use Collaborator for blind confirmation.
 - OWASP ZAP: Crawls and injects payloads into forms, detecting persistence.
 - Acunetix: Identifies stored issues via black-box testing.
 - SAST: SonarQube or Semgrep for code patterns like unsanitized database outputs.
- **Exploitation Tools:**

- XSSStrike or custom scripts for payload crafting.
- Browser extensions like XSS Hunter for monitoring triggers.

Recent disclosures include CVE-2025-1076 in Holded and CVE-2025-70849 in Podinfo, emphasizing unpatched risks.

Future:

By 2027, AI-generated code may amplify stored XSS by 25%, as models overlook sanitization in 70% of outputs. Trends favor AI-integrated scanners in CI/CD and stricter CSP adoption. However, with expanding user content in metaverses and IoT apps, new vectors could arise, balanced by auto-mitigating frameworks like Svelte or enhanced browser isolation.

Uses in Ethical Hacking:

Stored XSS is crucial for:

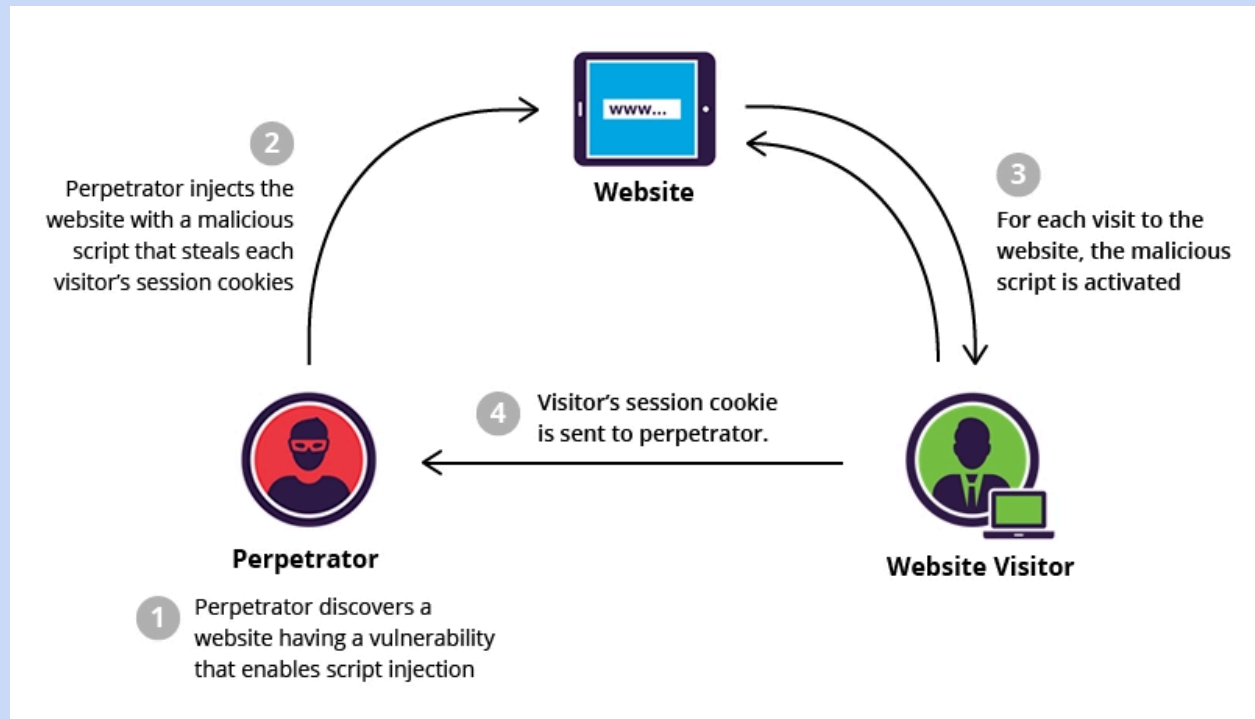
- **Pentesting:** Simulate real attacks to expose storage flaws; report with PoCs showing persistence.
- **Bug Bounties:** High rewards on platforms like HackerOne for impacts in admin areas.
- **Red Teaming:** Test social engineering by injecting payloads in shared resources.
- **Defensive Training:** Educate on input validation to prevent widespread breaches.

Ethical focus: Demonstrate harm potential without actual data theft.

Working Procedure:

Stored XSS operates via:

1. **Injection:** Attacker submits malicious input (e.g., `<script>alert('XSS')</script>`) via a form.
2. **Storage:** App saves it unsanitized in the database.
3. **Retrieval and Rendering:** Server fetches and includes the payload in responses without encoding.
4. **Execution:** Victim's browser loads the page, executing the script in the app's context.



Differences with Others:

Vulnerability Type	Stored XSS	Reflected XSS	DOM-based XSS
Location	Server-side storage and reflection	Server reflects in immediate response	Client-side DOM manipulation
Payload Delivery	Via forms; persisted in DB	Via URL params; one-off	Via client sources (e.g., hash); no server
Persistence	Persistent (affects many users)	Non-persistent	Non-persistent
Detection Difficulty	Low (scan stored data)	Medium (check responses)	High (JS analysis)
Examples	Malicious comment displayed	Echoed search query	<code>innerHTML</code> from URL
Mitigation Focus	Input validation + output encoding	Output encoding on reflections	Client safe APIs (e.g., <code>textContent</code>)

Example from DVWA lab:

The screenshot shows the DVWA (Damn Vulnerable Web Application) interface at the URL `localhost/dvwa/vulnerabilities/xss_s/`. The left sidebar contains a menu with various vulnerability categories, and the 'XSS (Stored)' option is highlighted in green. The main content area is titled 'Vulnerability: Stored Cross Site Scripting (XSS)'. It features a form with two input fields: 'Name *' and 'Message *'. Below the form are two buttons: 'Sign Guestbook' and 'Clear Guestbook'. The form is populated with the following data:

Name: test
Message: This is a test comment.

Name: shiva shan
Message: today is good day of my ethical hacking learning.M

Name: shiva shan
Message: today is good day of my ethical hacking learning.M

Below the form, there is a section titled 'More Information' with a list of links:

- <https://owasp.org/www-community/attacks/xss>
- <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <https://www.cgisecurity.com/xss-faq.html>
- <https://www.scriptalert1.com/>

The screenshot shows the DVWA (Damn Vulnerable Web Application) interface at the URL `localhost/dvwa/vulnerabilities/xss_s/`. The left sidebar contains a menu with various vulnerability categories, and the 'XSS (Stored)' option is highlighted in green. The main content area is titled 'Vulnerability: Stored Cross Site Scripting (XSS)'. It features a form with two input fields: 'Name *' and 'Message *'. Below the form are two buttons: 'Sign Guestbook' and 'Clear Guestbook'. The form is populated with the following data:

Name: shiva shan
Message: today is good day of my ethical hacking learning.M

Below the form, there is a section titled 'More Information' with a list of links:

- <https://owasp.org/www-community/attacks/xss>
- <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <https://www.cgisecurity.com/xss-faq.html>
- <https://www.scriptalert1.com/>

localhost/dvwa/vulnerabilities/xss_s/

Open P-TECH | Free... professional-intervi... GeeksforGeeks | A c... Support's School Projects - 1Stop X 1.1 Motivation and... X 1.1 Motivation and... Unsupervised Learn...

DVWA

Vulnerability: Stored Cross Site Scripting (XSS)

Name * shiva shan

Message * today is good day of my ethical hacking learning M

Sign Guestbook Clear Guestbook

Name: test
Message: This is a test comment.

More Information

- <https://owasp.org/www-community/attacks/xss>
- <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <https://www.cgisecurity.com/xss-faq.html>
- <https://www.scriptalert1.com/>

Elements Console Sources Network Performance Memory >>

```
<!DOCTYPE html>
<html lang="en-GB" suppresshydrationwarning="true" data-qb-installed="true">
<head>
</head>
<body class="home light" data-new-gr-c-s-check-loaded="14.1272.0" data-gr-ext-installed="data-new-gr-c-s-loaded="14.1272.0">
<div id="container">
  <div id="header"></div>
  <div id="main_menu"></div>
  <div id="main_body">
    <div class="body_padded">
      <h1>Vulnerability: Stored Cross Site Scripting (XSS)</h1>
      <div class="vulnerable_code_area">
        <form method="post" name="guestform">
          <table width="550" border="0" cellpadding="2" cellspacing="1">
            <tbody>
              <tr>
                <td width="100">Name *</td>
                <td>
                  <input name="txtName" type="text" size="30"
                    maxlength="100"> -- $0
                </td>
              </tr>
              <tr>
                <td width="100">Message *</td>
                <td>
                  <lt-mirror contenteditable="false" style="display: none;
                    "></lt-mirror>
                  <textarea name="txtMessage" cols="50" rows="3" maxlength=
                    "50" data-qb-tmp-id="lt-713420" spellcheck="false" data-
                    gramm="false"></textarea>
                </td>
              </tr>
            </tbody>
          </table>
        </form>
      </div>
    </div>
  </div>
</body>
</html>
```

localhost/dvwa/vulnerabilities/xss_s/

Open P-TECH | Free... professional-intervi... GeeksforGeeks | A c... Support's School Projects - 1Stop X 1.1 Motivation and... X 1.1 Motivation and... Unsupervised Learn...

DVWA

Vulnerability: Stored Cross Site Scripting (XSS)

Name * shiva shankar raja kumar vara prasad

Message * today is good day of my ethical hacking learning M

Sign Guestbook Clear Guestbook

Name: test
Message: This is a test comment.

More Information

- <https://owasp.org/www-community/attacks/xss>
- <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <https://www.cgisecurity.com/xss-faq.html>
- <https://www.scriptalert1.com/>

Elements Console Sources Network Performance Memory >>

```
<div id="main_body">
  <div class="body_padded">
    <h1>Vulnerability: Stored Cross Site Scripting (XSS)</h1>
    <div class="vulnerable_code_area">
      <form method="post" name="guestform">
        <table width="550" border="0" cellpadding="2" cellspacing="1">
          <tbody>
            <tr>
              <td width="100">Name *</td>
              <td>
                <input name="txtName" type="text" size="30" maxlength="100">
              </td>
            </tr>
            <tr>
              <td width="100">Message *</td>
              <td>
                <lt-mirror contenteditable="false" style="display: none;
                  "></lt-mirror>
                <textarea name="txtMessage" cols="50" rows="3"
                  maxlength="50" data-qb-tmp-id="lt-713420" spellcheck="fa
                  lse" data-gram="false"></textarea> -- $0
                </td>
              </tr>
            </tbody>
          </table>
        </form>
      </div>
    </div>
  </div>
  <div id="guestbook_comments"></div>
  <div>More Information</div>
</div>
</div>
</div>
```


localhost/dvwa/vulnerabilities/xss_s/

Open P-TECH | Free...professional-intervi...GeeksforsGeeks | A c...Support's SchoolProjects - 1StopX 1.1 Motivation and...X 1.1 Motivation and...Unsupervised Learni...

DVWA

HomeInstructionsSetup / Reset DBBrute ForceCommand InjectionCSRFFile InclusionFile UploadInsecure CAPTCHASQL InjectionSQL Injection (Blind)Weak Session IDsXSS (DOM)XSS (Reflected)XSS (Stored)CSP BypassJavaScript AttacksAuthorisation Bypass

Vulnerability: Stored Cross Site Scripting (XSS)

Name *

Message *

Sign GuestbookClear Guestbook

Name: test
Message: This is a test comment.

Name: shiva shan
Message: today is good day of my ethical hacking learning M

Name: shiva shan
Message: today is good day of my ethical hacking learning M

Name: shiva shakar raja kumar vara prasad
Message: today is good day of my ethical hacking learning. My day went of learning stored xss. which is very important in cybersecurity.

Name:
Message:

shlvakumar

shlvakumar son of shiva shankar

> (ctrl)|i to turn on code suggestions. Don't show againNEW